PSynergy

An Independent Journal for PlayStation 2 Linux Developers and Enthusiasts

Issue 1, February 15, 2004.

Table of Contents

EDITORIAL **2**

THIS MONTH IN THE FORUMS **3**

PROJECT SPOTLIGHT **5**

FEATURES **7**

FEATURE: PLAYSTATION2 IN HIGHER EDUCATION (SO YOU WANT TO BE A GAMES PROGRAMMER?) 7
FEATURE: GAMEFRAMES – GETTING YOU STARTED IN GAME DEVELOPMENT 13

PS2 DEVELOPMENT TIPS **16**

NEXT MONTH IN PSYNERGY **20**

End-Reader License

You my freely reproduce and redistribute this publication **in its entirety** through any means.

You may freely redistribute any article contained herein **in its entirety** as long as the name of the original author and the text "as per The Psynergy Journal" are placed both at the beginning and the end of the article. Also, this license paragraph must be copied and appended to the article to ensure that it continues to apply to the redistributed article.

Article Submission Guidelines

You may submit a copy of your article manuscript to patryk@pakl.net. For your article to even be considered, it must be related to development for the Playstation 2 Linux kit. Please note that this includes programming for the RTE, so long as anyone with the Kit can learn from/use it. All submissions will be edited for content, clarity and brevity and the author is given at least 3 days to approve final copy. A suggested template for articles will be provided at http://psynergy.pakl.net/ template.html. Finally, by submitting your article, you agree that it can be redistributed freely, only in its entirety, as long as your name is placed at the beginning and the end of it.

"Sony" is a registered trademark of Sony, Inc. "PlayStation", "PlayStation2", the PlayStation "PS" logo, and all associated logos, are trademarks of Sony Computer Entertainment, Inc. "PSynergy" and is NOT associated in ANY way with Sony Computer Entertainment, Inc.

Editorial

Welcome to the first official issue of PSynergy!!

First things first: PSynergy now has a website at http://psynergy.pakl.net. Code snippets associated with or referenced from articles can be easily accessed at the site (so that you do not have to cut and paste from a PDF file).

We have a series starting this month called GameFrames where Paul Smith (Eratosthenes) provides a very basic yet functional and complete game under PS2 Linux. We'd really like to hear about any changes or enhancements you have done to any GameFrames.

If you haven't yet viewed the original sample issue of PSynergy, please do! Even though it was a proof-of-concept issue, There were some interesting tidbits in that one. You can find it on the PSynergy website.

There's no shortage of really neat projects out there, but are there any development teams at work? In this issue, Henry Fortuna's article on the Playstation2 in academia mentions how group collaboration is essential in the accomplishment of great projects. However, it seems that most PS2Linux development is done by individuals (albeit asking questions in forums and in #ps2linux/#sps2). The results are necessarily quite simple or small. Then again, the PS2 Linux kit has only been available for a few years...

We hope you enjoy this issue of PSynergy and contribute to the next one! Remember that PSynergy has rolling submission, and submitting even preliminary drafts is encouraged at all times.

Patryk Laurent
Editor-in-Chief, PSynergy
patryk@playstation2-linux.com

This Month in the Forums

A Monthly Column by Eratosthenes

Hello and welcome to what I hope will become a regular column in PSynergy, where I take a look through the previous month's forum activity and pick out my favourites

This month, there was an absolute travesty in the Developers forum, bordering on the criminal. Sparky made a lovely post about a project he's working on with Exoticorn, and one line from that post overtook the entire thread! I'm going to focus here on what got missed in that thread - the packet stitcher and the RGBA indexer.

Getting the code

Sps2demo is as yet unreleased, but the latest versions of code are stored in the ps2-l CVS repository. Don't worry if you've never used a CVS repository before, instructions are included on the site. Go here (http://playstation2-linux.com/cvs/?group_id=206) to read the instructions on obtaining the latest sps2demo code. On that page is a link that reads "Browse CVS repository" which gives you web access to the appropriate files.

The Packet Stitcher

[sps2demo/shared/dma.(h|cpp)])

As some of you may know, there is currently a small limitation in SPS2 that has to be worked around, namely that memory allocated by SPS2, whilst being continuous in virtual memory, is only physically continuous in 4k chunks. This means you have to have some kind of method to 'stitch' up the 4k chunks back into one large DMA chain.

This is where Exoticorn's packet stitcher comes in. The stitching class is in sps2demo/shared/dma.(h|cpp). A test piece of code that makes use of it to draw a single triangle is in sps2demo/test/main.cpp.

Firstly, when using the packet stitcher, you need not concern yourself with the 4k restriction, it's all handled invisibly by the stitching class. Secondly, you'll notice there's no code in the test file to count the number of QWords placed into the DMA chain - that's all handled for you as well.

The eagle eyed amongst you will notice that in the code the sets up the DMA chain, dynDma.end() comes after the data had been added to the chain, where in actual fact the END DMAtag comes before the data. This is another beauty of the packet stitcher, it enables you to stop worrying about the order in which a DMA chain is built, and frees your mind to continue coding :)

You'll probably notice some instances of an sps2 object around the code - that's because the sps2demo also includes a wrapper around the sps2 library. However, there's nothing particularly funky going on in the wrapper, so I'll let you explore that yourself :).

So, go ahead, download the **sps2demo** code from CVS, navigate to the test directory, make the demo there, and run it to see a coloured triangle. Then have a look in main.cpp and see how easily it was done.

The RGBA indexer

[sps2demo/posteffects/rgbaindexer.(h|cpp)]

Some nifty effects are available in sps2demo thanks to a piece of code called the RGBAindexer. I won't go into the details of how it works, as that is

definately Sparky's domain, but I want to give some idea of what it can be used for.

The RGBAindexer allows you to operate on the 4 channels (Red, Green, Blue, and Alpha) of an image separately. For instance, you may wish to average the RGB components in some fashion to end up with a greyscale image. You could also combine this with a depth test to fade to grey gradually at a distance. Dot3 products also benefit from being able to reference components separately.

There is a lot of posteffect code in the sps2demo/posteffects folder. Unfortunately I can't lay my hands on some simple sample code to use straight away in your project, but as this project progresses there's bound to be more development on these effects - pester sparky in #sps2 on irc.freenode.net for more information!

Other news in brief

Myrkraverk announced the resurrection of his MIPS/EE Assembly HOWTO. He wants to continue working on this HOWTO, so get in touch with him if you have any MIPS/EE Assembly questions! (http://myrkraverk.afraid.org/PlayStation/EmotionEngine/)

Gita1111 asked whether the kind of graphics seen in certain SSX3 sequences is achievable on the Linux kit. I maintained that it is very much possible, though not done quite yet. I pointed to the VUniverse Demo, and Sparky pointed to the SPS2 vspeed demo as indicators that it certainly is possible. This then turned into a "So, how do I get started?" thread, giving birth to an article elsewhere this issue...
(http://playstation2-linux.com/files/vudemocontest/vu_coding_contest_2003_scea_1.1.tgz)
(http://playstation2-linux.com/files/sps2/sps2dev-0.3.0a.tgz)

Sparky and Jean_deda had a discussion about triangle strip clipping performance, in which Sparky shed some light on his clipping techniques
(http://playstation2-linux.com/forum/message.php?msg_id=40474)

Patryk announced a searchable code snippet repository, which could be handy should Sparky et al. post any new code snipets in the forums for us to salivate over :)
(http://psynergy.pakl.net/snippets/?id=8)

Sparky posted a code snippet about evaluating sin/cos polynomials
(http://playstation2-linux.com/forum/message.php?msg_id=40977)

Whilst discussing missing and/or corrupted quads I wrote about double buffering schemes used to send data to the GS.
(http://playstation2-linux.com/forum/message.php?msg_id=40637)

Finally, there was a post about the relative performance of triangle strips and triangle lists. Whilst the difference is large, the greatest thing to come out of this thread was the well known saying: "Premature optimization is the root of all evil." Whilst a little planning can go a long way, there's no substitute for overcoming your fears of only being 98% efficient, writing some code, and finding out that those extra few milliseconds really didn't matter after all.
(http://playstation2-linux.com/forum/message.php?msg_id=40637)

Project Spotlight

Projects: HSFortuna's Tutorials, PS2 as a Network Analyzer, Updated SPS2 to be released

How to DMA?

A fantastic resource out there to illustrate as simply as possible how to perform a DMA transfer using the registers made available via SPS2 is in H.S. Fortuna's programming tutorials. There are also a bunch of other interesting tidbits available there, so check it out!
http://www.hsfortuna.pwp.blueyonder.co.uk/

Application: PS2 as a Network Analyzer

Searching the web, we found a paper entitled "[PDF]Converting a Sony Playstation 2 into a Network Analyzer" which made use of the PS2 without hard disk project by Patryk (Patryk Laurent) and Eratosthenes (Paul Smith).

http://www1.cs.columbia.edu/~sedwards/classes/2002/w4995-02/crystal-final.pdf

Updated SPS2 to be released

In case you missed it in the forum discussions, a new version of SPS2 is in the works with some extra features. Here are some excerpts from Sauce's post about it (For more details, check the posts at and around http://playstation2-linux.com/forum/message.php?msg_id=41293)

Enabling EI/DI instructions

As mentioned, this is good if you have a reason to disable interrupts for a brief period of time. I'd certainly use this feature at your own risk, but hey, that's really true of all of sps2.

Swtiching between user & supervisor mode

…the reason I added it was to allow isolating measurements through performance counters to your application only. I don't believe there is anything else running in

supervisor mode, so if you switch the system to supervisor mode you can have the counters only count supervisor mode cycles. That being said, I've observed slight discrepancies in merasurements between user and supervisor modes. This feature is also to be used at your own risk….

UCAB support for sps2 memory and most likely

In addition to mapping code as cached & uncached, you now have the option to map as uncached accelerated. This sometimes offers performance advantages over using uncached memory but doesn't require an sps2FlushCache(). However, getting the right performance out of UCAB is a bit tricky, YMMV.

Support for GSVNC built into libsps2util

If I end up incorporating this by default into libsps2util, it will basically turn "on" gsvnc screen dumping for all applications that use libsps2util. Other than some memory, this should not cause a noticable performance decrease if a vnc server isn't running on the ps2 and if there are no clients connected to it. The code is highly optimized however the vnc server could use a little optimization. It runs as a separate process so you could al ways drop the priority on it. This allows you to view the front, back, & z buffers from a vnc client on a PC as well as suspend & resume your application by PC keypresses.

Features

Feature: PlayStation2 in Higher Education (So you want to be a Games Programmer?)

Dr Henry S Fortuna (h.s.fortuna@abertay.ac.uk)
University of Abertay Dundee, Scotland UK

Introduction

This article provides an insight into how a modern computer games console such as the PlayStation2 can be used effectively as a teaching and learning tool on Computer Science based courses within University higher education. The examples presented are taken from the "BSc Computer Games Technology" course that runs at The University of Abertay Dundee (UK). The author is intimately involved with the design and management of this course and is responsible for the teaching of console game programming within the University.

Abertay has a long association with the Computer Games Industry and was the first UK University to run a course in Computer Games Technology. In 1997 Sony Computers Entertainment Europe (SCEE) donated a laboratory of PlayStation1 Net Yaroze consoles to the course and this was followed up more recently in 2003 with the donation of a laboratory of PlayStation2 Linux kits.

In this article it will be argued that the use of games consoles on appropriate undergraduate and postgraduate courses is an excellent inspirational and motivational vehicle for the teaching and learning of many aspects of Computer Science to the highest level. Games console programming also provided an excellent in-context teaching tool enhancing employment prospects for student within the games industry.

Background

The Computer Game has a long and distinguished relationship with Computing, Engineering and Science Departments within University Higher Education. It is widely recognised that the first computer game, Spacewar, was developed by a young student at MIT on a Digital PDP-1 minicomputer in the early 1960s. At that time computers were large and expensive, and very few people could obtain access to one. In fact, the only real exposure to computers was from within University Computer Science Departments.
Through the 1970's and 80's young entrepreneurs, many of whom were students, successfully produced and published computer games using small personal computers which had become affordable to the individual. The available technology and game content at that time meant that it was possible for small groups of enthusiasts, with very little financial backing, to successfully produce and market computer games. Many of these early programmers,

however, still learned their computer programming skills within University Computer Science Departments.
The days of the individual producing a fully featured market topping computer game are now gone. Modern computer games are complex products produced by multidisciplinary teams of people working in a cooperative and creative manner. As a rough estimate, it takes about 20 man-years to develop a new game title, this being well beyond the realistic endeavour and capability of any single individual.
The computer games industry itself is now mature and well established. It is highly unlikely that anyone will gain employment as a professional games programmer simply by learning how to program on their own. Recruitment officers are looking for individuals with a strong University Computer Science Degree coupled with a flair for creativity and a strong portfolio of work. It is to satisfy this demand for high calibre individuals that the "Computer Games Technology" study program at Abertay was developed. The courses equips students with the technical knowledge associated with many of the traditional aspects of Computer Science, set in the creative and entrepreneurial environment of the computer games industry. This scenario is found to be highly motivational and inspirational to the current and new generation of computer science students, is captivating their imagination and nurturing their enthusiasm.

Modern computer games are complex products produced by multidisciplinary teams of people working in a cooperative and creative manner.

In order to do justice to this new generation of "Computer Games Technology" students, modern equipment must be available to provide exposure to the skills and technology that are current within the industry and this is where products such as the PlayStation2 Linux Kit are invaluable. The kit provides access to a current, modern, market leading console that is rich in both technical design and content and ideally suited for motivating the young, hungry mind.

Games Consoles in Education

This section will outline some of the main ways that the PlayStation2 Linux kit can and is being used for teaching and learning on the "Computer Games Technology" course at Abertay. Students entering the first year do not normally have experience programming in the C/C++ language and introductory modules are provided in this area. At present, Visual Studio running on Windows PCs is used as the programming platform but using the PlayStation2 Linux kits is being considered. One issue raised by first year students relates to a lack of exposure to "games development" equipment within this year of the course and it is thought that learning to program on a PlayStation2 will enhance motivation. It would also allow an early introduction to games console programming, this being of benefit in later years of the course. Programming in the "open" environment of a Linux system also provides many advantages.

Students also study computer organisation and architecture during the first year. The Playstation2 is an

excellent machine to consider as a case study for such modules. All aspects of computer architecture and organisation can be investigated using the PlayStation2 such as the CPU, bus systems, memory, co-processors, video systems, assembly language, micro-code, memory caches, to name but a few. The Playstation2 hardware reference manuals supplied with the kit also provide an ideal reference for students. Such detailed references are often hard, if not impossible to obtain for other commercial systems.

Students are exposed to many programming techniques including high level program structure and organisation in C++ classes, through to low level CPU and Vector Unit assembly language.

In the second year students gain access to the PlayStation2 kits to develop small games programs. This environment is an excellent vehicle for learning many if not all aspects of computer programming. It fact, it is generally agreed that games programming is one of the most demanding areas of programming since it involves the integration of a wide range of high level skills and techniques into one single product. During these modules students study topics such as the structure and organisation of a games program, the internal structure and organisation of the console and the tools necessary to create and import media content for games. By the end of the second year students will understand how consoles are structured and organised and the methods and techniques that are necessary in order to program consoles effectively. Students are exposed to many programming techniques including high level program structure and organisation in C++ classes, through to low level CPU and Vector Unit assembly language.

Students entering the third year have a solid grounding in computer architecture and programming and it is from this background that the PlayStation2 is used to introduce issues surrounding the design and construction of a 3D game engine. During second year, student were exposed to game engines through the use of high level 3D APIs such as OpenGL and DirectX, this mainly being in the context of high level API programming. The PlayStation2 programming theme in the third year is 3D game engine design but at a low level of abstraction. Students will develop and use the mathematical routines that are necessary for implementing a 3D engine and generate code that will interact directly with 3D console hardware such as the vector units. By the end of the third year students will have created a small prototype 3D game engine and will understand the structure, organisation and use of modern 3D game engines that are developed and used by commercial games companies.
Students also undertake a Group project in the third year of the course where they work up a proposal for the design and construction of a prototype game. Students are free to choose an implementation platform for this product and the PlayStation2 Linux kit is an ideal choice for this work.
Forty percent of the fourth and final year of the course constitutes a major Honours project where students propose, undertake and evaluate a research topic

which is current within the games industry. Once again, the Playstation2 Linux kit can be an ideal platform for this purpose. The kit provides a current console which can be used for testing and evaluating, algorithms, techniques and ideas.

One final theme that the PlayStation2 Linux kit is well suited to exposing within Computer Science courses is that of network programming and gaming. The Playstation2 Linux kit is supplied with a 10/100 Base-T Ethernet interface network adaptor and a Linux operating system with full network support. Under Linux, the Berkeley Sockets API provides access to the TCP/IP protocol suite that is the backbone of the internet. The PlayStation2 Linux kit can therefore be used in the teaching of network theory and practice and more specifically in the design and implementation of network computer games. Using the kit it is possible for students to design and create network enabled computer games which have global access through the internet.

The Development Configuration

This final section describes the development configuration implemented within the PlayStation2 teaching laboratory at Abertay. Figure 1 shows a view of the laboratory consisting of 20 development stations with the inset showing the plaque unveiled by Paul Holman, Vice President of Technology SCEE, during the opening ceremony on 9th June 2003. A close-up picture and a schematic diagram of one of the development stations is shown in Figures 2 and 3 respectively.



Figure 1



Figure 2

Each station consists of the following equipment: PlayStation2 with Linux kit, Windows PC, two PS2 controllers, dual input LCD monitor, keyboard, mouse, television and a peripheral connection box. The dual input LCD monitor is used to display the video output from either the PC or the PlayStation2. The single keyboard and mouse are switchable between the PC and the PS2 using a commercial interconnection box and suitable leads. Both the PC and the PS2 are connect to the University fast Ethernet network and communicate via the TCP/IP and Server Message Block protocols. Output from the PlayStation2 can be directed to the television when required. This arrangement significantly reduces the amount of equipment needed per station and maximised the utilisation of the laboratory.

Students are free to select a development method that is comfortable to them, but an arrangement found to be successful is as follows. The Linux file system is made available to the PC via a Samba server running on the PS2. Files on the PS2 can be created and edited using a suitable text editor (such as UltraEdit or Visual Studio) running on the PC. From either a Telnet or SSH session from the PC to the PS2, programs can be compiled and executed on the PS2 with graphics output being directed to either the LCD monitor or the television. Debug output from the program is sent via the Telnet/SSH session to a console window on the PC. Students can backup their files to the main University file servers making this arrangement a robust, effective and secure development environment.

Conclusions

This article has demonstrated how the PlayStation2 Linux kit can be applied to teaching and learning on a wide range of topics within Computer Science and Computer Games Technology courses. In practice, the kit has been found to be highly motivational for students and is an invaluable tool for the in-context teaching of Computer Games Technology.

I would finally like to stress that this article has only described the components of the Abertay course that relate to console game programming.

Dr Henry S Fortuna (h.s.fortuna@abertay.ac.uk)
Lecturer in Computer Games Technology
University of Abertay Dundee
Scotland UK

18 March 2004

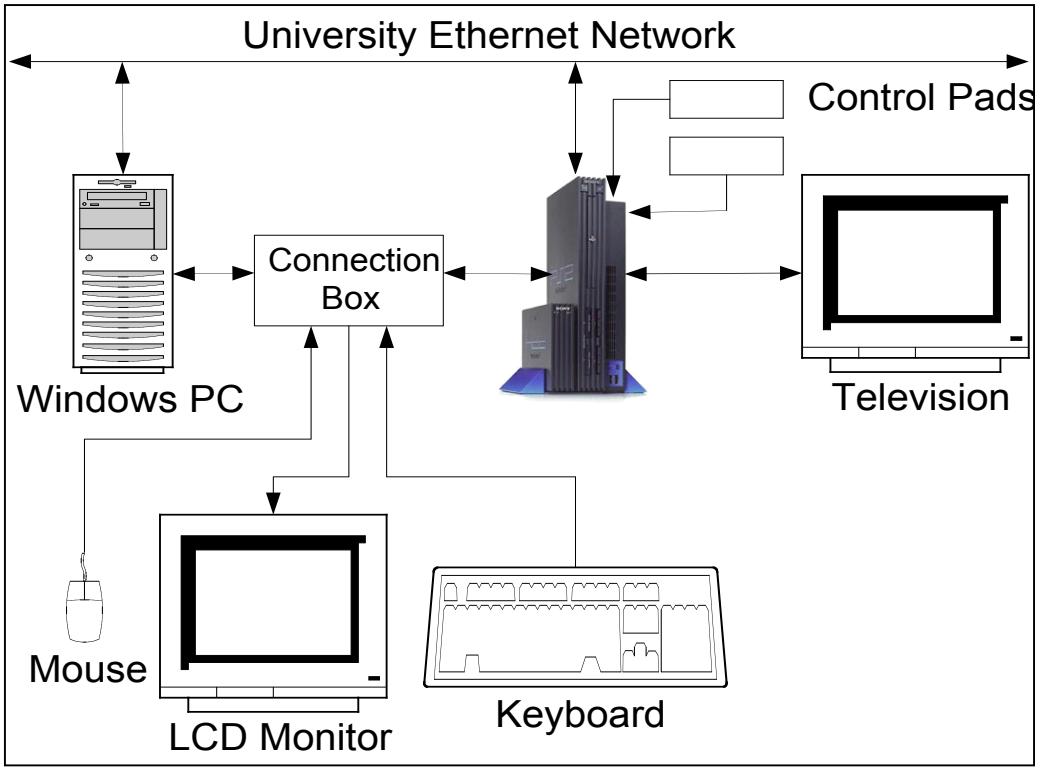


Figure 3

Features

Feature: GameFrames – Getting you started in Game Development

Eratosthemes (Paul Smith) (googoodolls_uk@yahoo.com)
School of Engineering, Computer Science and Mathematics
University of Exeter

What is a Gameframe?

If you're interested in game development, no doubt you've come across some really interesting articles about the latest techniques in AI, Collision Detection, Camera Control, the list is endless. If you're anything like me, you've wanted to try some of those ideas out, play with them, fiddle the details around and see what happens.

Unfortunately, to try those ideas out you normally need a working game to start with. That is what a GameFrame is - it's a fully functional game, with as little implemented as possible. What is implemented, however, is also documented. That way, you get:

- A working game to whittle away some of your free time
- Some working code that you should be able to read and digest
- Some ideas on what you can do to make it better, without having to start from scratch.

I'm hoping this will become a regular column, where I tackle different game genres to give you different starting points, and I also hope to be showcasing the best work that you guys produce!

Note: For those interested, the original code for this Pong clone actually came from Dr. H. S. Fortuna's website at http://www.hsfortuna.pwp.blueyonder.co.uk/ - Look for the 6th lesson entitled "*Simple game*"

GNOP -- Or "How much fun can you have with 13 boxes?"

This month, I'll tackle the veritable Behemoth that is Pong. Everyone knows what Pong is, everyone knows how to play it. All I've done is make a very basic implementation of it for the PS2, and make a list of what improvements you can get your teeth into.

http://psynergy.pakl.net/gameframes/01/

```
void GameLogic(void)
{
      CollisionDetectionAndResponse();
      DoControls();
      UpdateBall();
      counter++;
}
```

GNOP = "GNOP's not *only* Pong"

A lot of you will be wondering what the fuss is over a simple game of Pong? Well, test your programming skills on these tasks (in no order other than the order I thought of them in):

Potential Improvements

Variable ball movment/speed
If players are too good at the initial speed settings, the ball needs to be sped up as the game increases. More worryingly, the ball comes off the bat at

a totally predictable angle every time, which doesn't make for a very good game.

Variable controls
It's always nice to allow a player to use their own choice of buttons to control the game, but in this case it should be possible to allow 2 players to play from a single controller, as some people do not have a second controller. Currently the game exits with a harsh error message if 2 controllers are not found.

Analogue controls (pressure + sticks)
Part of the above section really - analogue controls are sometimes better than digital controls, and the player should be given a choice. For instance, as currently set up, the player must hold X to move faster, why introduce this extra button when you could merely have the player push the D-pad harder, which some may argue is more natural?

```
if (BoxCollision(Box[BALL], Box[LEFT_BAT]) ||
BoxCollision(Box[BALL],Box[RIGHT_BAT]))
{
  // Collision occured between ball and
  // a bat so turn ball around
  //Note - doesn't matter which bat.
  Box[BALL].xvel = -Box[BALL].xvel;
}
```

Title screen
Again, it's often polite to wait until players are ready before starting a game. Whilst the one-second delay helps, a title screen would help greatly here. I strongly doubt anyone could possibly have grown up without hearing of pong, or without instantly realising how to play the game, it would also be polite to have some kind of "how to play" demo feature in the intro.

End game
To comlpement the title screen, the game shouldn't just end when one player has lost 3 lives. We should first inform the players who has won, and then we should allow the choice of whether to play another game or not before exiting.

1-player mode
How about a computer player to play against? This would be quite difficult, as we need to create an AI that plays well, but has enough imperfections to be considered beatable. We could have a series of opponents to beat, each with different names, or we could simply pick a difficulty level.

Different play styles
I've put together a very simple version of pong here, but there are several well-known variants out there, such as soccer (where each player has 2 bats, effectively one striker and one goalkeeper). How about giving each player 2 bats, one which moves vertically, the other horizontally, and give one player the task of defending both the top and left walls? How about having targets behind the bat that need to be knocked down? Or have a goal that shrinks when your losing, to give the losing player a potentially crucial advantage?

Story mode?
Think of your favourite big budget game, and most of them have some kind of story mode. How about introducing some pong based challenges, and awarding the player medals for completing them, a certain number of which unlock new challenges and playing styles?

On-line play?
OK, I know it's 'only' Pong, but why not test out your network coding skills by allowing the 2 players to be on different PS2's? This will obviously require 2 PS2s, or a willing friend, to be tested though...

More that I've missed…?

The point I'm trying to make is that the number of improvements you could make is absolutely huge, and this is 'only' Pong. Let your imagination run wild!

And don't forget - if you get your teeth into this mission and start to get a bit stuck, use IRC channel #sps2 on irc.freenode.net or ask a question on the Developers forum at playstation2-linux.com. Please send in your finished work, we'll showcase the best submissions and collect them together for future readers.

Happy coding!

The author wishes to thank Dr. H. S. Fortuna for his tutorial series, and valuable help in writing this article.

PS2 Development Tips

Tips: 1) Drawing a Circle, 2) Journalling vs. Read Only Filesystems, and 3) Path 3 Synchronization

Tip #1: Drawing a Circle

snowcrash: I know how to draw a 2D box and trinagle, but not 2D circle. Can someone tell how to do this using libps2dev?

myrkraverk: This can be done with a trifan, at least that's how I would do it.
You can start with an octagon and slowly increase the number of triangles in the circle, to see various effects. Depending on the size of the circle, you should get it acceptably smooth with 30 – 60 triangle, I guess. This is mostly about sitting down and do some trigonometry to calculate the vertices. I suggest precalculate them, rather than doing them each time you draw the circle.

eratosthenes: Let me just expand on what myrk is saying. The GS can draw triangles natively, i.e you can tell it to draw a triangle and it knows what to do - but it doesn't knwo what a circle is. There are two ways I can think of to draw one - depending on what you want to do. One is to use the trifan primitive as myrk suggests, the other (which is fine if you're doing 2D work) is to texture map a sprite.

Using TriFans:
How do you split a circle up into lots of triangles? Put a point in the middle and draw equally spaced lines to the circumference - just like a pizza.
A TriFan works like this. You specify a list of vertices (points), and a set of triangles are drawn. Imagine a dot to dot pattern, if you told the GS to draw a TriFan with vertices (1, 2, 3, 4, 5) then it would draw 3 triangles, joining points (1, 2, 3) (1, 3, 4) and (1, 4, 5).
So, the first point you specify is a point in every triangle. This makes it a perfect candidate to specify the centre of the circle. Then you specify equally spaced points on the edge of a circle.
To do that, you need the sin and cos functions. To cut a long story short, a point at an angle alpha above the horizontal, on a circle radius r has co-ordinates:

```
x = r * cos(alpha) + xCentre;
y = r * sin(alpha) + yCentre;
```

So now, you can build up your triangle fan with a function something like this:

```
void drawCircle(int radius, int noOfPoints, int xCentre, int
yCentre)
{
SetGifTag(TRIFAN, EOP, ...);
//Place first point - centre of circle
addVertex(xCentre, yCentre);
float angle = 0.0f;
for(int i=0; i<=noOfPoints; i++) //<= so that circle is completed
{
      //Space equally around the circle (IN RADIANS!);
      angle += (2*PI) / noOfPoints;

      //Convert into x,y co-ords
      x = radius * cosf(angle) + xCentre;
      y = radius * sinf(angle) + yCentre;
      addVertex(x,y);
}
kickOffGifPacket;
}
```

Hope you get the gist of it. bjt uses TriFans to draw his asteroids in Plasmaroids - so if you're coding in VCL check out his source code for some pointers.

Using Texture mapping:
It might be easier to draw a sprite with a texture mapped onto it, previously having drawn a texture with a circle on it in Paint or similar, and make sure you get the outer region transparent using alpha.

HTH!

Eratosthenes

sparky: If you're into drawing circles then you simply must check out Foleys "Computer Graphics Principles and Practice", the chapter on rasterization. There are 2-3 pages explaining a killer simple and fast technique to do them by. Cosine and Sine are obviously not used.

Tip #2: Don't have journalling installed? Sauce suggests read-only

Just for the record... When I was doing hard-core development of sps2 (and crapping out my machine a LOT), I mounted most of my drives as read only.

For example, playing with COP0 right now seems to be crashing my machine often as well. I have a script that I use which does the following for me:

make
mount /home -o ro,remount
mount /usr -o ro,remount
./unload
sync
./load
And then I'm good to go. I play in the / file system which still ends up needing to be checked but is very small.

Note that I'm not condoning storing your files in the / file system. It's just a tricky directory to mount as read-only (there are often open files somewhere in my / file system, probably in /tmp), but I use it as a temporary fix when I'm doing stuff that's likely to crash my machine often.

Sauce

Tip #3: Path 3 Synchronization Technique

case123 (Luke): Hi there. Maybe i am missing something fundamental, but why is the 24 * 4 NOP's between FLUSHA;FLUSH;MSKPATH(3) & MSKPATH(0x8000) ? ie. why 24 ? thanks in advance.

sparky: Actually I think it's the best question you could have asked about this sample. It's basicly the 1 million dollar question. Why is there always room for gello and why is it 24 quads of nops. Noone at scee knows and noone at scei will tell them since they've moved on to bigger and better things I'm told. The man in pro support who has been trying to make this technique work since a while back has basicly had to guess the right size through trial and error. Noone at scee knows if it was even the intention to do texture syncronization using mskpath3 or if it was intended for something more innocent. However it has proven to be a very effective technique.
The only intuition there is to back up that he might have hit the size of the button is:
1. it works
2. vif FIFO is 16quads and during the round robin trip of the DMA the delivery size per channel is 8 quads (16+8 = 24).
If you ask him this is what he'll tell you but he'll also agree it doesn't actually prove it.

The idea ofcourse is that vif opens path3 using MSKPATH3(0) then the vif needs to wait just long enough for the first giftag (EOP=0) of the next texture delivery to be received by the gif and then it can close up again afterwards using MSKPATH3(0x8000). By leaving every giftag of the texture with an EOP of zero except the last one, you will postpone the lockup of path3 to after the entire texture has been transfered. It will then wait until you open it again from vif1.
Don't forget to check out the intermittent bit, that's something that's used for a quicker turnaround time. The bit is set on the GIF_MODE register, you'll see that in screen.cpp in the setup function at the top.
The idea is you only want it to upload texture while path1 isn't being used such as when VU is using it's thinking hat and rasterization of the previous batch is already complete. Then it's a good time to upload, enabling IMT bit makes sure the rights are returned to path1 once it's ready for transfer. When IMT is on this check is done at every 8 quads, if IMT is NOT on then it will continue transfer until completion and VU can't get away with even a single kick during the period.
I hope this was useful to you.

Hikey: A very complete and honest answer. Although the 24 qwords don't have to be NOPs, they can be actual VIF codes or data, like register settings.

Sparky: I forgot to mention that, thanx ;) I always just use nops myself but you can use whatever you like to use while waiting. In my opinion the waiting (24Q nops) are however a very small fraction of the time used during the rendering, so I've never bothered to fill them up myself. keep us posted case ;)

Next Month in PSynergy

Look for Psynergy Next Month:

- New articles!

- A thorough look at optimal DMA to and from the VPUs under SPS2 (based on/inspired by excerpts from the discussion between blevine and sparky as found in the forums http://playstation2-linux.com/forum/message.php?msg_id=37440)

- Another installement of GameFrames!

- Whatever else comes into the inbox over here!